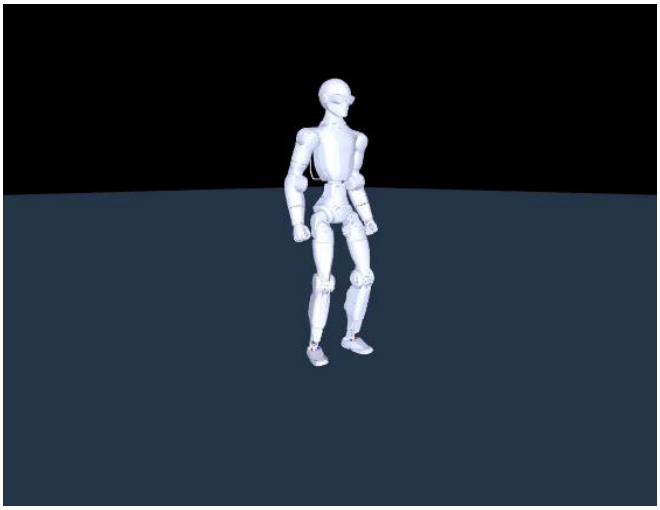
Embodied AI

Lab Manual for MuJoCo

Lab 2 — Teach It To Stand

September 2026



Shaoqin Li

Applied and Embodied AI Lab
Electrical Engineering Department
The City College of New York

you build every file yourself · one 2 MB download · your training runs in the background

Contents

Hardware: any laptop, no GPU · **Before you start:** finish Lab 1. **You start from your Lab 1 folder.** No new archive of code is handed to you. You create every file in this lab the same way you did in Lab 1 — by making it yourself and running it to prove it works. Exactly one thing is downloaded, and Part 1 explains why it cannot be typed.

> **Copy the code from the `.md` version of this manual, not from the PDF.** You were given both. Open `LAB2_teach_it_to_stand.md` in VS Code beside this document and copy from there — copying Python out of a PDF drops indentation, and indentation *is* Python.
>
> Every `Expected` block below was produced by actually running the command shown, on a machine built by following Lab 1 and then this manual. **If your numbers differ, that is a finding — tell your instructor.** Verified 2026-09-01 on Ubuntu 24.04 under WSL2, MuJoCo 3.12.0, PyTorch 2.14.0+cpu, stable-baselines3 2.9.0.

The shape of this lab is different from Lab 1

Lab 1 was *run and observe*. Nothing took longer than six minutes.

This lab contains **training**, which takes about an hour. So the session works like this:

1. You build the files and **launch** the training — Parts 0 to 4.
2. Your run keeps going in the background while you work through the rest.
3. You are given a **finished policy** to observe immediately — no waiting.
4. At the end, you come back to your own run and see how far it got.

Nothing you need depends on your training finishing. If you want the full run, leave it going at home — you already have every file.

What you will do

Part		What you end up with
0	Two more libraries	PyTorch and stable-baselines3
1	Build the `exp2` workspace	folders, 43 meshes, one download
2	Change the robot's actuators	`model/r1_walk_train.xml`
3	Create the environment	`r1_walk_env.py` — the task itself
4	**Launch your training** — then leave it alone	a run that grows all session
5	Read what you just launched	what the policy senses and controls
6	Read the reward	how you say "good" without saying "how"
7	See a finished policy	pictures, and a window if your machine allows
8	Does it stand?	`exp2_eval.py`, and a number
9	What is it actually paid for?	the answer to an objection you raise first
10	Push it	the same shove that beat Lab 1
11	Back to your own run	`export_policy.py`, and the lesson of the lab

What you should be able to say afterwards

1. Why a learned policy can recover from a shove when a PD controller structurally cannot.
2. What the reward is actually paying for — and how to check rather than assume.
3. **Why a rising training curve does not mean your policy works.**
4. What changed in the robot file between Lab 1 and Lab 2, and why it had to change.

Part 0 — Two more libraries

Everything from Lab 1 still applies: same Ubuntu, same conda, same `r1lab` environment. Open a terminal and activate it:

```
conda activate r1lab
```

Two additions. The first is **PyTorch**, which trains the network. On Linux the default wheel drags in about 4.8 GB of CUDA that a laptop cannot use, so ask for the CPU build by name:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

Expected — the last line:

```
Successfully installed MarkupSafe-3.0.3 filelock-3.32.3 jinja2-3.1.6 mpmath-1.3.0 networkx-3.6.1
sympy-1.14.0 torch-2.14.0+cpu
```

The second is **stable-baselines3**, the reinforcement-learning library that implements PPO:

```
pip install stable-baselines3
```

Expected — the last line:

```
Successfully installed cloudpickle-3.1.2 farama-notifications-0.0.6 gymnasium-1.3.0
stable-baselines3-2.9.0
```

Check both landed:

```
python - <<'EOF'
import torch, stable_baselines3, gymnasium
print("torch              ", torch.__version__)
print("stable-baselines3  ", stable_baselines3.__version__)
print("gymnasium          ", gymnasium.__version__)
EOF
```

Expected:

```
torch              2.14.0+cpu
stable-baselines3  2.9.0
gymnasium          1.3.0
```

> ☑ **Checkpoint: three version numbers, and `torch` says `+cpu`.** If it does not say `+cpu` you downloaded the CUDA build — it still works, it just cost you 4.8 GB.

> **Only Parts 4, 8, 9, 10 and 11 need these two.** Parts 5, 6 and 7 run on `mujoco` and `numpy` alone. If your install fails, do not stop — keep going and come back to it.

Part 1 — Build the `exp2` workspace

Lab 1 lives in `~/r1_lab/exp1`. This lab gets its own folder beside it, and **borrows the 43 meshes you already downloaded** — you are not fetching the robot from GitHub a second time.

```
mkdir -p ~/r1_lab/exp2/model ~/r1_lab/exp2/policies ~/r1_lab/exp2/runs
cd ~/r1_lab/exp2
cp -r ~/r1_lab/exp1/model/assets model/assets
ls model/assets | wc -l
```

Expected:

```
43
```

Every command in this lab runs from `~/r1_lab/exp2`. Open it in the editor now:

```
code .
```

Directory	Holds

Directory	Holds
`~/r1_lab/exp2`	this experiment — every script you write
`exp2/model`	the robot description and its 43 shapes
`exp2/policies`	trained policies in a plain, portable format
`exp2/runs`	training runs, yours and the reference one

The one thing you download

Everything else in this lab you type. **A trained policy cannot be typed** — it is a few hundred thousand numbers that came out of an hour of computation. Your instructor gives you `lab2_policy.tar.gz` (2 MB). Extract it into the folder you just made:

```
tar -xzf ~/Downloads/lab2_policy.tar.gz -C ~/r1_lab/exp2
```

```
ls policies runs/exp2_stand
```

Expected:

```
policies:
exp2_stand.npz

runs/exp2_stand:
best.zip
vecnormalize_best.pkl
```

Three files, and they are three different things:

File	What it is	Who reads it
`policies/exp2_stand.npz`	the trained network as plain arrays	`watch_policy.py` — **no PyTorch needed**
`runs/exp2_stand/best.zip`	the same policy as a PyTorch checkpoint	`exp2_eval.py`
`runs/exp2_stand/vecnormalize_best.pkl`	the input statistics that go with it	`exp2_eval.py`

> **Why two copies of one policy?** Because a network is only useful with the exact scaling its inputs were trained with, and the two formats carry it differently. The `.npz` is a stripped export — 45 → 256 → 256 → 12 weights plus the observation mean and variance — which any machine with NumPy can run. In Part 11 you make one of these yourself, from your own run.

> ☑ **Checkpoint: 43 meshes and three downloaded files.**

Part 2 — Change the robot's actuators

Your Lab 1 robot stood by holding one pose with a stiff spring at every joint: `kp = 600`. That is not how the real R1 is driven, and it is not how a learned policy drives it.

Start from the file you already wrote:

```
cp ~/r1_lab/exp1/model/r1_standalone.xml model/r1_walk_train.xml
```

Open `model/r1_walk_train.xml` in VS Code and find the `<actuator>` block near the bottom. It is 14 lines: an opening tag, twelve `<position>` entries — `kp="600"` at the hips and knees, `kp="300"` at the ankles, each with `dampratio="1"` — and a closing tag. **Delete all 14 lines**, opening and closing tag included, and paste this in their place:

```
  <actuator>
    <position name="left_hip_pitch" joint="left_hip_pitch_joint" kp="150" kv="8" />
    <position name="left_hip_roll" joint="left_hip_roll_joint" kp="150" kv="8" />
    <position name="left_hip_yaw" joint="left_hip_yaw_joint" kp="150" kv="8" />
    <position name="left_knee" joint="left_knee_joint" kp="200" kv="10" />
    <position name="left_ankle_pitch" joint="left_ankle_pitch_joint" kp="60" kv="4" />
    <position name="left_ankle_roll" joint="left_ankle_roll_joint" kp="60" kv="4" />
    <position name="right_hip_pitch" joint="right_hip_pitch_joint" kp="150" kv="8" />
    <position name="right_hip_roll" joint="right_hip_roll_joint" kp="150" kv="8" />
    <position name="right_hip_yaw" joint="right_hip_yaw_joint" kp="150" kv="8" />
    <position name="right_knee" joint="right_knee_joint" kp="200" kv="10" />
    <position name="right_ankle_pitch" joint="right_ankle_pitch_joint" kp="60" kv="4" />
    <position name="right_ankle_roll" joint="right_ankle_roll_joint" kp="60" kv="4" />
    <position name="waist_roll" joint="waist_roll_joint" kp="80" kv="5" />
    <position name="waist_yaw" joint="waist_yaw_joint" kp="80" kv="5" />
    <position name="left_shoulder_pitch" joint="left_shoulder_pitch_joint" kp="60" kv="4" />
    <position name="left_shoulder_roll" joint="left_shoulder_roll_joint" kp="60" kv="4" />
    <position name="left_shoulder_yaw" joint="left_shoulder_yaw_joint" kp="40" kv="3" />
    <position name="left_elbow" joint="left_elbow_joint" kp="40" kv="3" />
    <position name="left_wrist_roll" joint="left_wrist_roll_joint" kp="40" kv="3" />
    <position name="right_shoulder_pitch" joint="right_shoulder_pitch_joint" kp="60" kv="4" />
    <position name="right_shoulder_roll" joint="right_shoulder_roll_joint" kp="60" kv="4" />
    <position name="right_shoulder_yaw" joint="right_shoulder_yaw_joint" kp="40" kv="3" />
    <position name="right_elbow" joint="right_elbow_joint" kp="40" kv="3" />
    <position name="right_wrist_roll" joint="right_wrist_roll_joint" kp="40" kv="3" />
  </actuator>
```

Save. Three things changed, and each one has a reason.

1. Twelve actuators became twenty-four. Lab 1 drove the legs and let the arms hang. Here the waist and arms are actively held at a fixed pose, exactly as the real robot's controller holds them while its legs do the work. Arms that swing freely change the balance problem.

2. Lab 1's gains became the real robot's gains. Lab 1 held every leg joint with `kp = 600` (300 at the ankles) and let MuJoCo pick the damping from `dampratio="1"`. Here each joint gets the stiffness and damping the Unitree controller actually applies — 150/8 at the hips, 200/10 at the knees, 60/4 at the ankles. Lab 1 was free to pick any spring it liked because nothing outside the simulator cared. From here on it matters: **a policy trained against the wrong stiffness cannot be moved to the real machine.**

3. Nothing else. Not one body, mass, mesh, joint limit or contact changed. Check for yourself:

```
diff ~/r1_lab/exp1/model/r1_standalone.xml model/r1_walk_train.xml
```

Expected — one hunk, and nothing else:

```
365,376c365,388
<     <position name="left_hip_pitch"    joint="left_hip_pitch_joint"    kp="600" dampratio="1"/>
<     <position name="left_hip_roll"     joint="left_hip_roll_joint"     kp="600" dampratio="1"/>
...
```

Twelve lines out, twenty-four in, at line 365 — and not one line changed anywhere else in 390. The opening and closing `<actuator>` tags are identical in both files, which is why the count is 12/24 and not 14/26.

Now confirm the file compiles and carries what you think it does:

```
python - <<'EOF'
import mujoco
m = mujoco.MjModel.from_xml_path("model/r1_walk_train.xml")
print("actuators   ", m.nu)
print("total mass   %.2f kg" % sum(m.body_mass))
for i in (0, 3, 4, 12, 14):
    n = mujoco.mj_id2name(m, mujoco.mjtObj.mjOBJ_ACTUATOR, i)
    print("  %-20s kp=%-5g kv=%g" % (n, m.actuator_gainprm[i][0], -m.actuator_biasprm[i][2]))
EOF
```

Expected:

```
actuators    24
total mass   28.93 kg
  left_hip_pitch       kp=150   kv=8
  left_knee            kp=200   kv=10
  left_ankle_pitch     kp=60    kv=4
  waist_roll           kp=80    kv=5
  left_shoulder_pitch  kp=60    kv=4
```

28.93 kg — the same robot as Lab 1. Only the way it is driven changed.

> ☑ **Checkpoint: 24 actuators, 28.93 kg.** If you get 12 actuators, the old block is still there. If the file will not compile, the paste is truncated — `wc -l model/r1_walk_train.xml` must say **390**.

Part 3 — Create the environment

Lab 1's script was a loop you wrote: set the target, step the physics, measure. Training needs the same thing wrapped in a standard interface, so that a learning library can drive it without knowing anything about robots. That interface is a **Gymnasium environment**, and this is the file that defines the entire task: what the policy sees, what it controls, when an episode ends, and what counts as good.

It is the longest file in the workshop — 471 lines. You are not expected to read all of it now; Parts 5 and 6 walk you through the parts that matter. Create `r1_walk_env.py` and paste it in.

```
"""Gymnasium walking environment for Unitree R1 in plain MuJoCo.

Trains a velocity-tracking walking policy for the 12 leg joints. Arms and waist
are PD-held at the stand pose (matching the unitree_mujoco deployment), so the
policy only outputs 12 leg position-target offsets.

The observation is built ONLY from quantities the real DDS pipeline also
exposes (rt/lowstate): base angular velocity (gyro), projected gravity (from
the base quaternion), the velocity command, leg joint positions/velocities, and
the previous action. Base linear velocity is used for the reward but is NOT in
the observation (not reliably available on hardware) -- this keeps the obs
identical between training and sim2sim.

Control: policy at 50 Hz (decimation 10 over the 500 Hz physics), low-level PD
handled by the position actuators whose gains match the deployment torque PD.
"""
import os
import numpy as np
import mujoco
import gymnasium as gym
from gymnasium import spaces

HERE = os.path.dirname(os.path.abspath(__file__))
# default to the original model; set R1_WALK_XML=r1_walk_train_matched.xml to
# train on the deployment-physics-matched model (shrinks the sim2sim gap).
_xml_name = os.environ.get("R1_WALK_XML", "r1_walk_train.xml")
# Look beside this file first, then in a model/ subfolder. The student lab keeps
# its models in model/, the research tree keeps them alongside the code.
XML = next((c for c in (os.path.join(HERE, _xml_name),
                        os.path.join(HERE, "model", _xml_name),
                        _xml_name) if os.path.exists(c)),
           os.path.join(HERE, _xml_name))

# Leg actuator/joint order (indices 0..11), matches build_xml.py LEG_ACTS.
LEG_JOINTS = [
```

```
    "left_hip_pitch_joint", "left_hip_roll_joint", "left_hip_yaw_joint",
    "left_knee_joint", "left_ankle_pitch_joint", "left_ankle_roll_joint",
    "right_hip_pitch_joint", "right_hip_roll_joint", "right_hip_yaw_joint",
    "right_knee_joint", "right_ankle_pitch_joint", "right_ankle_roll_joint",
]
# Default (stand) leg pose, same values as r1_stand_sim2sim.py STAND_POS legs.
DEFAULT_LEG = np.array([-0.1, 0, 0, 0.3, -0.2, 0,
                        -0.1, 0, 0, 0.3, -0.2, 0], dtype=np.float64)
# Arm/waist hold targets (actuator order 12..23, matches ARM_WAIST_ACTS).
ARM_WAIST_HOLD = np.array([0, 0, 0.35, 0.18, 0, 0.87, 0,
                           0.35, -0.18, 0, 0.87, 0], dtype=np.float64)

FOOT_GEOMS = [f"{s}_foot{i}_collision" for s in ("left", "right") for i in range(1, 8)]

DECIMATION = 10          # 500 Hz physics -> 50 Hz control
ACTION_SCALE = 0.25
EP_LEN_S = 20.0

# obs scales
S_ANGVEL = 0.25
S_DOFVEL = 0.05

# ---- domain randomization ranges (per episode unless noted) ----
# The sim2sim gap is a DYNAMICS gap (training capsule contacts vs unitree_mujoco
# full-mesh + sphere-marker feet, plus motor/contact mismatch). These ranges
# build a robustness margin so the policy survives that gap instead of overfitting
# to one model. Applied only when dr=True.
DR = dict(
    friction=(0.5, 1.4),        # sliding friction, floor + foot geoms
    body_mass=(0.90, 1.12),     # per-body mass/inertia scale
    trunk_mass=(0.85, 1.25),    # extra scale on the pelvis (payload/battery slop)
    kp_scale=(0.85, 1.15),      # motor stiffness (models actuator mismatch)
    kv_scale=(0.85, 1.15),      # motor damping
    # Pushes address EXTERNAL-disturbance robustness, which is not the sim2sim
    # failure mode (nothing pushes the robot in the transfer test). Keep them mild
    # so episodes run long enough for the DYNAMICS randomizations above -- the ones
    # that actually close the training->unitree_mujoco gap -- to be learned.
    push_mag=0.3,               # max per-axis push velocity (m/s)
    push_period_s=(2.5, 4.0),   # push interval, randomized
)
# per-step observation noise std (raw units, added before obs scaling)
OBS_NOISE = dict(angvel=0.05, gravity=0.02, dofpos=0.01, dofvel=0.15)

# Control latency (in 50Hz control steps) randomized per episode. THIS is the
# real sim2sim gap: the DDS harness round-trip acts like ~1-3 steps of delay, and
# the zero-margin trained gait can't tolerate even 20-40ms. Training up to MAX_LATENCY
# forces a latency-robust (more grounded) gait. Verified: 2-step delay reproduces the
# ~2.7s unitree_mujoco fall on the exact deployment physics.
MAX_LATENCY = 4

# OPTION 3 (2026-07-11): walk4_lat trained against a FIXED per-episode latency
# (one uniform(0,4) draw held for the whole 20s episode) and got WORSE on the
# real harness, not better -- because measured DDS behavior is NOT a constant
# offset. measure_dds.py found: mean obs staleness 0.7ms (<<1 control step),
```

```
# with occasional publish-dt spikes to ~20ms (1 control step) from OS
# scheduling/GC, and the observed real fall matches a 2-step (40ms) spike.
# So the correct training signal is PER-STEP intermittent spikes on top of a
# near-zero baseline, not a per-episode constant delay. Opt-in via jitter_dr=True.
JITTER_DR = dict(
    spike_prob=0.04,        # P(a given control step is a "spike" step)
    spike_latency=(1, 3),   # spike magnitude range (control steps) when triggered
    rare_stall_prob=0.003,  # P(a rare larger GC-pause-like stall)
    rare_stall_latency=(4, 8),
)

# reward weights
# --- OPTION 2 additions (2026-07-11): the push baseline showed the gait
# already has good margin against EXTERNAL disturbance (88% full-survival
# under pushes) -- the actual gap is CONTROL-DELAY margin (60%/high-variance
# at just 2 control steps = 40ms, per r1_walk_latency_sweep.py). Classical
# control theory: delay margin trades off against closed-loop bandwidth,
# and bandwidth/reactivity is roughly what a lower CoM + wider support
# polygon buys mechanically (more passive stability = less reliant on fast
# active correction, which is exactly what stale/delayed commands break).
RW = dict(
    lin_vel=1.5, ang_vel=0.5, alive=0.15,
    lin_vel_z=-2.0, ang_vel_xy=-0.05, orientation=-5.0,
    torque=-1e-4, action_rate=-0.01, dof_acc=-2.5e-7,
    base_height=-10.0, feet_air_time=1.0, feet_slide=-0.1,
    heading=-1.5,    # penalize yaw deviation from the start heading (kills circle-walking)
    lateral=-1.0,    # penalize sideways (body-y) velocity (kills crabbing)
    stance_width=-4.0,  # OPTION 2: penalize lateral foot separation BELOW target (wider support
polygon)
)
TARGET_H = 0.65          # OPTION 2: lowered from 0.70 -- more crouched = lower m*g*h
                          # toppling stiffness (see teaching notes Sec 2), mechanically
                          # more stable per unit disturbance/delay, at the cost of some
                          # torque efficiency (acceptable trade for delay-margin).
TARGET_STANCE_WIDTH = 0.22  # nominal lateral foot separation (m) to reward toward


def yaw_from_quat(q):
    w, x, y, z = q
    return np.arctan2(2.0 * (w*z + x*y), 1.0 - 2.0 * (y*y + z*z))


def wrap_pi(a):
    return (a + np.pi) % (2.0 * np.pi) - np.pi


def projected_gravity(q):
    """Gravity vector [0,0,-1] expressed in the base frame = R(q)^T @ [0,0,-1].
    Equals minus the third row of R(q); written out to avoid building a matrix."""
    w, x, y, z = q
    return np.array([-2.0*(x*z - w*y),
                     -2.0*(y*z + w*x),
                     -(w*w - x*x - y*y + z*z)])
```

```
class R1WalkEnv(gym.Env):
    metadata = {"render_modes": []}

    def __init__(self, cmd_range=(0.2, 0.6), push=True, dr=True, seed=None,
                 fixed_cmd=None, fixed_latency=None, jitter_dr=False,
                 zero_rew=None):
        super().__init__()
        # EXPERIMENT 3: reward-shaping ablation.  Names listed here are computed
        # as usual (so the UNMODIFIED total is still reported in info["full_rew"]
        # and can be used to score every ablation on one fixed yardstick) but are
        # dropped from the reward the policy actually optimizes.  Scoring an
        # ablated policy by its own ablated objective would be circular --
        # deleting a penalty raises the score by construction.
        self.zero_rew = set(zero_rew or [])
        unknown = self.zero_rew - set(RW)
        if unknown:
            raise ValueError(f"unknown reward term(s) {sorted(unknown)}; "
                             f"valid: {sorted(RW)}")
        self.model = mujoco.MjModel.from_xml_path(XML)
        self.model.opt.timestep = 0.002
        self.data = mujoco.MjData(self.model)
        self.rng = np.random.default_rng(seed)
        self.cmd_range = cmd_range
        self.push = push
        self.dr = dr
        self.fixed_cmd = None if fixed_cmd is None else np.asarray(fixed_cmd, float)
        # force an EXACT control-delay (in 50Hz control steps) every episode,
        # bypassing DR's randomized 0..MAX_LATENCY draw -- for measuring the
        # delay-margin curve (survival vs injected latency) precisely.
        self.fixed_latency = fixed_latency
        # OPTION 3: per-step intermittent spike model instead of a per-episode
        # constant latency draw -- see JITTER_DR comment above.
        self.jitter_dr = jitter_dr
        self.dt = self.model.opt.timestep * DECIMATION
        self.max_steps = int(EP_LEN_S / self.dt)

        m = self.model
        self.leg_qadr = np.array([m.jnt_qposadr[mujoco.mj_name2id(
            m, mujoco.mjtObj.mjOBJ_JOINT, j)] for j in LEG_JOINTS])
        self.leg_vadr = np.array([m.jnt_dofadr[mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_JOINT,
j)] for j in LEG_JOINTS])
        # all 24 joint qpos/qvel adr for reset (leg + arm order == actuator order)
        all_joints = LEG_JOINTS + [
            "waist_roll_joint", "waist_yaw_joint",
            "left_shoulder_pitch_joint", "left_shoulder_roll_joint", "left_shoulder_yaw_joint",
            "left_elbow_joint", "left_wrist_roll_joint",
            "right_shoulder_pitch_joint", "right_shoulder_roll_joint",
"right_shoulder_yaw_joint",
            "right_elbow_joint", "right_wrist_roll_joint",
        ]
        self.all_qadr = np.array([m.jnt_qposadr[mujoco.mj_name2id(
            m, mujoco.mjtObj.mjOBJ_JOINT, j)] for j in all_joints])
        self.default_all = np.concatenate([DEFAULT_LEG, ARM_WAIST_HOLD])
```

```
        self.gyro_adr = self._sensor_adr("imu_ang_vel")
        self.linvel_adr = self._sensor_adr("imu_lin_vel")
        self.floor_gid = mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_GEOM, "floor")
        # discover foot collision geoms from the model (robust to the capsule model
        # with 7/foot and the deployment-matched model with 1 box/foot)
        all_gnames = [mujoco.mj_id2name(m, mujoco.mjtObj.mjOBJ_GEOM, i) for i in range(m.ngeom)]
        foot_names = [g for g in all_gnames if g and "foot" in g and "collision" in g]
        self.foot_gids = [mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_GEOM, g) for g in foot_names]
        self.left_gids = {gid for gid, g in zip(self.foot_gids, foot_names) if
g.startswith("left")}
        self.right_gids = {gid for gid, g in zip(self.foot_gids, foot_names) if
g.startswith("right")}

        self.nominal_h = self._compute_nominal_height()

        # pristine copies of everything DR mutates, so each reset randomizes from
        # the nominal model rather than compounding previous episodes' scaling.
        self.orig_body_mass = m.body_mass.copy()
        self.orig_body_inertia = m.body_inertia.copy()
        self.orig_gainprm = m.actuator_gainprm.copy()
        self.orig_biasprm = m.actuator_biasprm.copy()
        self.orig_geom_friction = m.geom_friction.copy()
        self.pelvis_bid = mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_BODY, "pelvis")
        if self.pelvis_bid < 0:   # fall back to the base body (child of world/free joint)
            self.pelvis_bid = int(m.jnt_bodyid[0]) if m.njnt else 1
        # OPTION 2: ankle body ids for the stance-width reward
        self.left_ankle_bid = mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_BODY,
"left_ankle_roll_link")
        self.right_ankle_bid = mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_BODY,
"right_ankle_roll_link")

        self.action_space = spaces.Box(-1.0, 1.0, (12,), np.float32)
        n_obs = 3 + 3 + 3 + 12 + 12 + 12  # angvel, gravity, cmd, dofpos, dofvel, prev_action
        self.observation_space = spaces.Box(-np.inf, np.inf, (n_obs,), np.float32)

        self.prev_action = np.zeros(12)
        self.command = np.zeros(3)
        self.air_time = np.zeros(2)
        self.prev_leg_vel = np.zeros(12)
        self.steps = 0
        self._push_every = int(2.0 / self.dt)

    def _sensor_adr(self, name):
        sid = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_SENSOR, name)
        return self.model.sensor_adr[sid]

    def _compute_nominal_height(self):
        """Set base z so the feet rest on the floor at the stand pose."""
        d = mujoco.MjData(self.model)
        d.qpos[self.all_qadr] = self.default_all
        d.qpos[2] = 1.0
        mujoco.mj_forward(self.model, d)
        min_fz = min(d.geom_xpos[g][2] for g in self.foot_gids)
```

```
        # geom center z; foot capsule radius 0.01 -> lowest point ~min_fz-0.01
        return 1.0 - min_fz + 0.01

    def _apply_dr(self):
        """Randomize model dynamics from the pristine nominal each reset."""
        m = self.model
        rng = self.rng
        # --- mass + inertia (per-body scale, extra slop on the pelvis) ---
        mass_scale = rng.uniform(*DR["body_mass"], size=m.nbody)
        mass_scale[self.pelvis_bid] *= rng.uniform(*DR["trunk_mass"])
        m.body_mass[:] = self.orig_body_mass * mass_scale
        m.body_inertia[:] = self.orig_body_inertia * mass_scale[:, None]
        # --- sliding friction: floor + both feet (independent) ---
        m.geom_friction[:] = self.orig_geom_friction
        m.geom_friction[self.floor_gid, 0] = rng.uniform(*DR["friction"])
        for g in self.foot_gids:
            m.geom_friction[g, 0] = rng.uniform(*DR["friction"])
        # --- PD gains (position actuator: gainprm[0]=kp, biasprm[1]=-kp, [2]=-kv) ---
        kp_s = rng.uniform(*DR["kp_scale"]) * rng.uniform(0.96, 1.04, size=m.nu)
        kv_s = rng.uniform(*DR["kv_scale"]) * rng.uniform(0.96, 1.04, size=m.nu)
        m.actuator_gainprm[:] = self.orig_gainprm
        m.actuator_biasprm[:] = self.orig_biasprm
        m.actuator_gainprm[:, 0] = self.orig_gainprm[:, 0] * kp_s
        m.actuator_biasprm[:, 1] = self.orig_biasprm[:, 1] * kp_s
        m.actuator_biasprm[:, 2] = self.orig_biasprm[:, 2] * kv_s

    # -----------------------------------------------------------------
    def reset(self, *, seed=None, options=None):
        if seed is not None:
            self.rng = np.random.default_rng(seed)
        d = self.data
        mujoco.mj_resetData(self.model, d)
        d.qpos[self.all_qadr] = self.default_all + self.rng.uniform(-0.03, 0.03, 24)
        d.qpos[2] = self.nominal_h + self.rng.uniform(-0.01, 0.01)
        # small random yaw
        yaw = self.rng.uniform(-0.1, 0.1)
        d.qpos[3:7] = [np.cos(yaw/2), 0, 0, np.sin(yaw/2)]
        d.qvel[:] = 0
        self.yaw0 = yaw          # heading to hold while walking straight

        if self.dr:
            self._apply_dr()
            # random control latency 0..MAX_LATENCY steps (models the DDS round-trip;
            # this is the dominant sim2sim gap -- see MAX_LATENCY note)
            self.latency = int(self.rng.integers(0, MAX_LATENCY + 1))
            self._push_every = max(1, int(self.rng.uniform(*DR["push_period_s"]) / self.dt))
        else:
            self.latency = 0
        if self.fixed_latency is not None:
            self.latency = int(self.fixed_latency)   # override, for delay-margin measurement

        mujoco.mj_forward(self.model, d)
        self.prev_action[:] = 0
        self.prev_leg_vel[:] = d.qvel[self.leg_vadr]
```

```
        jitter_max = JITTER_DR["rare_stall_latency"][1] if self.jitter_dr else 0
        buf_len = max(MAX_LATENCY, self.latency, jitter_max) + 1
        self.act_buf = [np.zeros(12) for _ in range(buf_len)]  # control-latency delay
        self.air_time[:] = 0
        self.steps = 0
        # sample command (forward-biased) unless a fixed command was set
        if self.fixed_cmd is not None:
            self.command = self.fixed_cmd.copy()
        else:
            vx = self.rng.uniform(*self.cmd_range)
            self.command = np.array([vx, 0.0, 0.0])
        return self._obs(), {}

    def _obs(self):
        d = self.data
        angvel = d.sensordata[self.gyro_adr:self.gyro_adr+3].copy()
        grav = projected_gravity(d.qpos[3:7])
        dofpos = d.qpos[self.leg_qadr] - DEFAULT_LEG
        dofvel = d.qvel[self.leg_vadr].copy()
        if self.dr:
            # sensor noise -> robustness to the deployment's noisier obs
            n = OBS_NOISE
            angvel += self.rng.normal(0, n["angvel"], 3)
            grav = grav + self.rng.normal(0, n["gravity"], 3)
            dofpos = dofpos + self.rng.normal(0, n["dofpos"], 12)
            dofvel = dofvel + self.rng.normal(0, n["dofvel"], 12)
        obs = np.concatenate([
            angvel * S_ANGVEL,
            grav,
            self.command,
            dofpos,
            dofvel * S_DOFVEL,
            self.prev_action,
        ]).astype(np.float32)
        return obs

    def _foot_contacts(self):
        d = self.data
        lc = rc = False
        for i in range(d.ncon):
            c = d.contact[i]
            g1, g2 = c.geom1, c.geom2
            pair = {g1, g2}
            if self.floor_gid in pair:
                other = g1 if g2 == self.floor_gid else g2
                if other in self.left_gids:
                    lc = True
                elif other in self.right_gids:
                    rc = True
        return np.array([lc, rc])

    def step(self, action):
        action = np.clip(action, -1.0, 1.0).astype(np.float64)
        # control latency: apply the action from `latency` control-steps ago
```

```
        self.act_buf.append(action)
        jitter_max = JITTER_DR["rare_stall_latency"][1] if self.jitter_dr else 0
        buf_len = max(MAX_LATENCY, self.latency, jitter_max) + 1
        if len(self.act_buf) > buf_len:
            self.act_buf.pop(0)
        # OPTION 3: per-step intermittent spike, drawn fresh each control step
        # (not held constant for the episode like the original DR/fixed_latency).
        step_latency = self.latency
        if self.jitter_dr:
            u = self.rng.random()
            if u < JITTER_DR["rare_stall_prob"]:
                step_latency = int(self.rng.integers(*JITTER_DR["rare_stall_latency"]))
            elif u < JITTER_DR["rare_stall_prob"] + JITTER_DR["spike_prob"]:
                step_latency = int(self.rng.integers(*JITTER_DR["spike_latency"]))
            else:
                step_latency = 0
        applied = self.act_buf[-(step_latency + 1)]
        leg_target = DEFAULT_LEG + ACTION_SCALE * applied
        ctrl = np.concatenate([leg_target, ARM_WAIST_HOLD])
        self.data.ctrl[:] = ctrl

        for _ in range(DECIMATION):
            mujoco.mj_step(self.model, self.data)

        self.steps += 1
        d = self.data

        # random push (bigger + randomized interval when DR is on)
        if self.push and self.steps % self._push_every == 0:
            pm = DR["push_mag"] if self.dr else 0.3
            d.qvel[0:2] += self.rng.uniform(-pm, pm, 2)

        # --- reward terms ---
        linvel = d.sensordata[self.linvel_adr:self.linvel_adr+3]     # base frame
        angvel = d.sensordata[self.gyro_adr:self.gyro_adr+3]
        grav = projected_gravity(d.qpos[3:7])
        base_z = d.qpos[2]
        leg_vel = d.qvel[self.leg_vadr]
        leg_force = d.actuator_force[:12]

        r = {}
        dvx = linvel[0] - self.command[0]; dvy = linvel[1] - self.command[1]
        r["lin_vel"] = RW["lin_vel"] * np.exp(-(dvx*dvx + dvy*dvy) / 0.25)
        wz_err = (angvel[2] - self.command[2])**2
        r["ang_vel"] = RW["ang_vel"] * np.exp(-wz_err / 0.25)
        r["alive"] = RW["alive"]
        r["lin_vel_z"] = RW["lin_vel_z"] * linvel[2]**2
        r["ang_vel_xy"] = RW["ang_vel_xy"] * (angvel[0]**2 + angvel[1]**2)
        r["orientation"] = RW["orientation"] * (grav[0]**2 + grav[1]**2)
        r["torque"] = RW["torque"] * float(leg_force @ leg_force)
        da = action - self.prev_action
        r["action_rate"] = RW["action_rate"] * float(da @ da)
        acc = (leg_vel - self.prev_leg_vel) / self.dt
        r["dof_acc"] = RW["dof_acc"] * float(acc @ acc)
```

```
        r["base_height"] = RW["base_height"] * (base_z - self.nominal_h)**2
        # keep a straight heading + no crabbing (fixes the reward-hacked curved gait)
        yaw_err = wrap_pi(yaw_from_quat(d.qpos[3:7]) - self.yaw0)
        r["heading"] = RW["heading"] * yaw_err * yaw_err
        r["lateral"] = RW["lateral"] * linvel[1] * linvel[1]

        # OPTION 2: reward lateral foot separation toward TARGET_STANCE_WIDTH (wider
        # base of support = more margin before the CoM projection exits the support
        # polygon under a stale/delayed correction). Only reward reaching the target
        # or beyond -- no bonus (or penalty) for going wider than target.
        stance_w = abs(d.xpos[self.left_ankle_bid][1] - d.xpos[self.right_ankle_bid][1])
        shortfall = max(0.0, TARGET_STANCE_WIDTH - stance_w)
        r["stance_width"] = RW["stance_width"] * shortfall ** 2

        # feet air time (encourages stepping, not shuffling)
        contacts = self._foot_contacts()
        first_contact = contacts & (self.air_time > 0)
        self.air_time += self.dt
        air_reward = np.sum((self.air_time - 0.4) * first_contact)
        self.air_time[contacts] = 0.0
        # only reward stepping when actually commanded to move
        cmd_mag = np.linalg.norm(self.command[:2])
        r["feet_air_time"] = RW["feet_air_time"] * air_reward * (cmd_mag > 0.1)

        # EXPERIMENT 3: full_reward is the unmodified objective -- kept for
        # scoring; `reward` is what this policy is actually trained on.
        full_reward = sum(r.values())
        for k in self.zero_rew:
            r[k] = 0.0
        reward = sum(r.values())

        self.prev_action = action.copy()
        self.prev_leg_vel = leg_vel.copy()

        # termination
        fell = base_z < 0.4
        tilted = grav[2] > -0.5
        bad = not np.isfinite(d.qpos).all() or not np.isfinite(d.qvel).all()
        terminated = bool(fell or tilted or bad)
        truncated = self.steps >= self.max_steps
        if terminated:
            reward -= 5.0
            full_reward -= 5.0

        tilt_deg = float(np.degrees(np.arccos(np.clip(-grav[2], -1.0, 1.0))))
        info = {"rew": r, "full_rew": full_reward, "cmd": self.command.copy(),
                "fwd_vel": float(linvel[0]), "base_z": float(base_z),
                "base_xy": (float(d.qpos[0]), float(d.qpos[1])),
                "tilt_deg": tilt_deg, "max_leg_torque": float(np.max(np.abs(leg_force)))}
        obs = self._obs() if not bad else np.zeros(self.observation_space.shape, np.float32)
        return obs, float(reward), terminated, truncated, info


if __name__ == "__main__":
```

```
    # quick self-test
    env = R1WalkEnv(seed=0)
    print("nominal_h", round(env.nominal_h, 4), "max_steps", env.max_steps, "dt", env.dt)
    o, _ = env.reset()
    print("obs dim", o.shape, "act dim", env.action_space.shape)
    tot = 0.0
    for i in range(200):
        o, rew, term, trunc, info = env.step(env.action_space.sample() * 0.0)  # hold stand
        tot += rew
        if term or trunc:
            print("episode end at", i, "term", term, "trunc", trunc)
            break
    print("held-stand 200-step return:", round(tot, 2), "final base_z", round(info["base_z"], 3))
```

Prove it builds and produces the right shapes:

```
python - <<'EOF'
from r1_walk_env import R1WalkEnv
e = R1WalkEnv(push=False, dr=False, fixed_cmd=[0, 0, 0], seed=0)
o, _ = e.reset()
print(o.shape[0], "in /", e.action_space.shape[0], "out")
EOF
```

Expected:

```
45 in / 12 out
```

> ☑ **Checkpoint: 45 in, 12 out.** A `ModuleNotFoundError: gymnasium` means Part 0 did not finish.

Part 4 — Launch your training

Do this **now**, before you understand it, so that it accumulates while you work through the rest of the lab. You will come back to it in Part 11.

Create `r1_walk_train.py`:

```
"""PPO training for the R1 walking policy (single-env, CPU -- GTX 1650 laptop).

Usage:
  python3 r1_walk_train.py --steps 10000000 --name run1
  python3 r1_walk_train.py --steps 30000    --name smoke   # smoke test

Saves checkpoints (model + VecNormalize stats) under runs/<name>/ every
--save-freq steps, plus tensorboard logs. Resumable-ish: latest checkpoint +
```

```
vecnormalize can be loaded by the eval / sim2sim scripts.
"""
import os
import argparse
import numpy as np

from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv, VecNormalize
from stable_baselines3.common.callbacks import BaseCallback

from r1_walk_env import R1WalkEnv

HERE = os.path.dirname(os.path.abspath(__file__))

try:
    import tensorboard  # noqa: F401
    _HAS_TB = True
except ImportError:
    _HAS_TB = False


class SaveCallback(BaseCallback):
    """Save model + VecNormalize stats periodically and track best ep reward."""
    def __init__(self, save_freq, save_dir, vecnorm):
        super().__init__()
        self.save_freq = save_freq
        self.save_dir = save_dir
        self.vecnorm = vecnorm
        self.best = -np.inf

    def _on_step(self):
        if self.n_calls % self.save_freq == 0:
            self.model.save(os.path.join(self.save_dir, "latest"))
            self.vecnorm.save(os.path.join(self.save_dir, "vecnormalize.pkl"))
            # keep a numbered snapshot too, so a good-behaviour policy is never
            # overwritten by a later higher-reward-but-worse one (lesson from walk1)
            k = self.num_timesteps // 1000
            self.model.save(os.path.join(self.save_dir, f"ckpt_{k}k"))
            self.vecnorm.save(os.path.join(self.save_dir, f"vn_{k}k.pkl"))
            # ep_rew_mean from the monitor
            if len(self.model.ep_info_buffer) > 0:
                mean_r = np.mean([e["r"] for e in self.model.ep_info_buffer])
                mean_l = np.mean([e["l"] for e in self.model.ep_info_buffer])
                if mean_r > self.best:
                    self.best = mean_r
                    self.model.save(os.path.join(self.save_dir, "best"))
                    self.vecnorm.save(os.path.join(self.save_dir, "vecnormalize_best.pkl"))
                print(f"[{self.num_timesteps}] ep_rew_mean={mean_r:.2f} "
                      f"ep_len_mean={mean_l:.0f} best={self.best:.2f}", flush=True)
        return True


def make_env(seed, jitter_dr=False, fixed_cmd=None, zero_rew=None):
    def _f():
```

```
        from stable_baselines3.common.monitor import Monitor
        return Monitor(R1WalkEnv(seed=seed, jitter_dr=jitter_dr, fixed_cmd=fixed_cmd,
                                 zero_rew=zero_rew))
    return _f


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--steps", type=int, default=10_000_000)
    ap.add_argument("--name", type=str, default="run1")
    ap.add_argument("--save-freq", type=int, default=50_000)
    ap.add_argument("--seed", type=int, default=0)
    ap.add_argument("--resume", action="store_true")
    ap.add_argument("--init-from", default=None, help="warm-start policy weights from this .zip")
    ap.add_argument("--init-vn", default=None, help="load obs-norm stats from this .pkl")
    ap.add_argument("--n-envs", type=int, default=1,
                    help="parallel envs via SubprocVecEnv (each gets seed+rank); "
                         "total rollout size stays 4096 (n_steps = 4096 // n_envs) "
                         "so parallelism/batch-diversity is the ONLY changed variable")
    ap.add_argument("--fixed-cmd", type=float, nargs=3, default=None,
                    help="hold the velocity command fixed, e.g. --fixed-cmd 0 0 0 "
                         "turns the walk task into a standing task")
    ap.add_argument("--jitter-dr", action="store_true",
                    help="OPTION 3: per-step intermittent latency spikes (JITTER_DR in "
                         "r1_walk_env.py) instead of/on top of the per-episode-constant "
                         "0..MAX_LATENCY draw already in dr=True")
    ap.add_argument("--zero-rew", type=str, default=None,
                    help="EXPERIMENT 3: comma-separated reward terms to drop from "
                         "the training objective, e.g. --zero-rew heading,lateral. "
                         "The unmodified reward is still logged as info['full_rew'] "
                         "so every ablation can be scored on one fixed yardstick.")
    args = ap.parse_args()
    zero_rew = [s.strip() for s in args.zero_rew.split(",")] if args.zero_rew else None

    save_dir = os.path.join(HERE, "runs", args.name)
    os.makedirs(save_dir, exist_ok=True)

    fns = [make_env(args.seed + i, jitter_dr=args.jitter_dr, fixed_cmd=args.fixed_cmd,
                    zero_rew=zero_rew)
           for i in range(args.n_envs)]
    venv = SubprocVecEnv(fns) if args.n_envs > 1 else DummyVecEnv(fns)
    vn_path = os.path.join(save_dir, "vecnormalize.pkl")
    if args.resume and os.path.exists(vn_path):
        venv = VecNormalize.load(vn_path, venv)
        venv.training = True
    elif args.init_vn and os.path.exists(args.init_vn):
        # warm start: carry obs-normalization stats from the source run;
        # reward norm re-adapts to the (changed) reward on its own.
        venv = VecNormalize.load(args.init_vn, venv)
        venv.training = True
        print("loaded obs-norm stats from", args.init_vn, flush=True)
    else:
        venv = VecNormalize(venv, norm_obs=True, norm_reward=True,
                            clip_obs=10.0, clip_reward=10.0, gamma=0.99)
```

```
    policy_kwargs = dict(net_arch=dict(pi=[256, 256], vf=[256, 256]))
    # keep the TOTAL rollout at 4096 steps regardless of n_envs, so multi-env
    # runs change batch diversity (independent trajectories), not batch size
    n_steps = max(4096 // args.n_envs, 256)
    override = {"n_steps": n_steps}
    model_path = os.path.join(save_dir, "latest.zip")
    if args.resume and os.path.exists(model_path):
        model = PPO.load(model_path, env=venv, device="cpu", custom_objects=override)
        print("resumed from", model_path, flush=True)
    elif args.init_from and os.path.exists(args.init_from):
        model = PPO.load(args.init_from, env=venv, device="cpu", custom_objects=override)
        print("warm-started policy weights from", args.init_from, flush=True)
    else:
        model = PPO(
            "MlpPolicy", venv, device="cpu", verbose=0,
            n_steps=n_steps, batch_size=256, n_epochs=5,
            gamma=0.99, gae_lambda=0.95, clip_range=0.2,
            ent_coef=0.004, learning_rate=3e-4, vf_coef=0.5,
            max_grad_norm=1.0, policy_kwargs=policy_kwargs,
            # Only log to tensorboard if it is actually installed. SB3 raises
            # ImportError at learn() time otherwise, which killed training on a
            # clean student install where tensorboard is not a dependency.
            tensorboard_log=(os.path.join(save_dir, "tb") if _HAS_TB else None),
        )

    cb = SaveCallback(args.save_freq, save_dir, venv)
    print(f"training '{args.name}' for {args.steps:,} steps -> {save_dir}", flush=True)
    print(f"  seed={args.seed}  n_envs={args.n_envs}  n_steps={n_steps}  "
          f"zero_rew={zero_rew or 'none (baseline)'}", flush=True)
    model.learn(total_timesteps=args.steps, callback=cb,
                reset_num_timesteps=not args.resume, progress_bar=False)
    model.save(os.path.join(save_dir, "latest"))
    venv.save(vn_path)
    print("done. saved to", save_dir, flush=True)


if __name__ == "__main__":
    main()
```

Then launch it:

```
python r1_walk_train.py --name my_run --fixed-cmd 0 0 0 --steps 3000000 \
    --n-envs 4 --save-freq 12500
```

Expected — a line every 50,000 steps:

```
training 'my_run' for 3,000,000 steps -> /home/YOURNAME/r1_lab/exp2/runs/my_run
  seed=0  n_envs=4  n_steps=1024  zero_rew=none (baseline)
[50000] ep_rew_mean=-28.59 ep_len_mean=59 best=-28.59
[100000] ep_rew_mean=-16.31 ep_len_mean=76 best=-16.31
```

Reward starts negative. That is correct — an untrained policy falls immediately and collects nothing but penalties. `ep_len_mean=59` means the average episode ended after 59 control steps, about 1.2 seconds. It is falling over, repeatedly, and being paid nothing for it.

Now leave it running and open a second terminal (`conda activate r1lab`, `cd ~/r1_lab/exp2`). Everything from here on happens in the new one.

milestone	steps	roughly
first stands reliably	1M	**25 min**
past a rough patch you will meet in Part 11	2M	50 min
full run	3M	74 min

(At about 673 steps/s on a busy laptop. A quiet machine reaches ~1,020, so yours may be faster.)

Three flags are worth understanding:

- `--fixed-cmd 0 0 0` is what makes this *standing*. The environment's task is "match the commanded velocity"; commanding zero means *hold still on purpose*. The reward, the physics and the observation stay byte-identical to every other experiment in this workshop, so later comparisons measure the policy and not a changed setup.
- `--n-envs 4` runs four copies of the robot in parallel, one per core. Same learning, four times the experience per second.
- `--save-freq 12500` — the checkpoint counter counts *batches*, not steps, and with 4 environments one batch is 4 steps. The default would give you about 15 checkpoints for the whole run. You want 60. **Part 11 depends on it.**

> ☑ **Checkpoint: a negative reward on the first line, and a new folder `runs/my_run`.**

Part 5 — Read what you just launched

What the policy sees — 45 numbers

You printed this shape in Part 3. Here is what the 45 numbers are:

Slice	What it is	Size
`[0:3]`	how fast the body is rotating	3
`[3:6]`	which way is down, from the robot's point of view	3
`[6:9]`	the command — all zeros in this lab	3
`[9:21]`	where each leg joint is, relative to the default pose	12
`[21:33]`	how fast each leg joint is moving	12
`[33:45]`	what it did on the previous step	12

Notice what is absent. No camera. No knowledge of where it is in the world. No base velocity — that one is deliberate: it is easy in simulation and unreliable on real hardware, so it is used in the *reward* but kept out of the *observation*. That is what would let this same policy run on a physical robot unchanged.

Find the line in `r1_walk_env.py` that builds this vector — search for `def _obs`. Every term in it is something the real R1 publishes over its own network interface. That is not an accident; it is the whole design constraint.

What it controls — 12 numbers

Twelve leg joint **position targets**, produced every 20 ms. This is the whole difference from Lab 1: in Lab 1 the target was a constant you chose. Here it is the output of a network, fifty times a second.

The twelve waist and arm actuators you added in Part 2 are held at a fixed pose by the environment, not by the policy. The policy never sees them.

Part 6 — Read the reward

The reward is how you say **good** without saying **how**.

```
python - <<'EOF'
from r1_walk_env import RW
for k, v in RW.items():
    print(f"{k:>16}  {v:>8}")
EOF
```

Expected:

```
         lin_vel       1.5
         ang_vel       0.5
           alive      0.15
       lin_vel_z      -2.0
      ang_vel_xy     -0.05
     orientation      -5.0
          torque   -0.0001
     action_rate     -0.01
         dof_acc  -2.5e-07
     base_height     -10.0
   feet_air_time       1.0
      feet_slide      -0.1
         heading      -1.5
         lateral      -1.0
    stance_width      -4.0
```

Fifteen terms. Three do nearly all the work:

Term	Weight	Asks for

Term	Weight	Asks for
`lin_vel`	**+1.5**	match commanded velocity → *hold still*
`ang_vel`	**+0.5**	match commanded turn rate → *do not spin*
`alive`	**+0.15**	a small bonus for every step not fallen
`orientation`	−5.0	stay upright
`base_height`	−10.0	hold hip height
`torque`	−1e-4	do not waste effort

Nowhere does any of this say *bend the knee*, *put that foot there*, or *lean back when shoved*. Lab 1's controller was told the pose. This one is told only the goal.

The objection you should raise now, before any result

The moment there is an "alive" bonus, there is an obvious criticism:

> *"Your robot did not learn to balance. It learned to collect the survival bonus."*

That is a fair objection, and **no training curve can answer it.** Reward going up is consistent with both stories. Hold onto it — you answer it yourself in Part 9, with a measurement.

Part 7 — See a finished policy

Your own run is nowhere near ready. Here is one that is — the reference policy you downloaded in Part 1, trained for 3M steps by exactly the command you launched in Part 4.

Two tools. The first needs no PyTorch and no window; the second needs a window.

7a — Watch it, headless *(works everywhere)*

Create `watch_policy.py`. This is the viewer for the rest of the workshop — Lab 3 uses it too.

```
"""Watch a TRAINED policy drive the R1 — Labs 2 and 3.

Needs only mujoco + numpy. The policy is a plain .npz (a 45->256->256->12 MLP
plus its observation statistics), exported from the trained network, so there is
no PyTorch and no stable-baselines3 to install.

    python watch_policy.py                                  # Lab 2: it stands
    python watch_policy.py --push 0.30                      # Lab 2: survives
    python watch_policy.py --push 0.40                      # Lab 2: falls
    python watch_policy.py --noise gravity 10               # Lab 3: blind it
    python watch_policy.py --zero command                   # Lab 3: the null control
    python watch_policy.py --latency 8                      # Lab 3: 160 ms delay
    python watch_policy.py --policy walk3b.npz --vx 0.5     # a walking policy
    python watch_policy.py --headless                       # verdict only, no window
```

```
KEYS: left-drag orbit · scroll zoom · space pause · Esc quit
"""
import argparse, os, sys, time
import numpy as np
import mujoco

HERE = os.path.dirname(os.path.abspath(__file__))

LEG = ["left_hip_pitch", "left_hip_roll", "left_hip_yaw", "left_knee",
       "left_ankle_pitch", "left_ankle_roll", "right_hip_pitch", "right_hip_roll",
       "right_hip_yaw", "right_knee", "right_ankle_pitch", "right_ankle_roll"]
DEFAULT_LEG = np.array([-0.1, 0, 0, 0.3, -0.2, 0, -0.1, 0, 0, 0.3, -0.2, 0])
ARM_HOLD = np.array([0, 0, 0.35, 0.18, 0, 0.87, 0, 0.35, -0.18, 0, 0.87, 0])
ACTION_SCALE, DECIMATION = 0.25, 10
S_ANGVEL, S_DOFVEL = 0.25, 0.05
# observation slices and the noise each channel saw in training
# channel -> (where it sits in the 45-dim observation, the noise it saw in
# TRAINING, the scale already applied to it).  --noise doses in multiples of that
# training sigma, so "x10" means ten times what the policy was trained to expect.
#
# sigma=None means the channel had no injected training noise, so there is no
# natural unit.  For those we dose in multiples of the channel's own training
# standard deviation, read from the policy's obs_var.  Do not invent a fixed
# number here: the command channel's training sd is 5.9e-06, so ANY absolute
# noise is amplified ~170,000x by normalisation and saturates the clip bound.
# That is why --noise command is not a null control.  Use --zero for that.
CH = {"angvel": (slice(0, 3), 0.05, S_ANGVEL), "gravity": (slice(3, 6), 0.02, 1.0),
      "dofpos": (slice(9, 21), 0.01, 1.0), "dofvel": (slice(21, 33), 0.15, S_DOFVEL),
      "command": (slice(6, 9), None, 1.0), "prev_action": (slice(33, 45), None, 1.0)}


def load_policy(path):
    d = np.load(path)
    def act(obs):
        o = np.clip((obs - d["obs_mean"]) / np.sqrt(d["obs_var"] + d["epsilon"]),
                    -d["clip_obs"], d["clip_obs"])
        h = np.tanh(d["w0"] @ o + d["b0"])
        h = np.tanh(d["w1"] @ h + d["b1"])
        return np.clip(d["w2"] @ h + d["b2"], d["act_low"], d["act_high"])
    return act


def projected_gravity(q):
    w, x, y, z = q
    return np.array([-2*(x*z - w*y), -2*(y*z + w*x), -(w*w - x*x - y*y + z*z)])


class Robot:
    def __init__(self, xml, vx):
        self.m = mujoco.MjModel.from_xml_path(xml)
        self.m.opt.timestep = 0.002
        self.d = mujoco.MjData(self.m)
        self.m.vis.headlight.ambient[:] = [0.5, 0.5, 0.5]
```

```
        self.m.vis.headlight.diffuse[:] = [0.6, 0.6, 0.6]
        name2id = lambda n: mujoco.mj_name2id(self.m, mujoco.mjtObj.mjOBJ_JOINT, n + "_joint")
        self.qadr = np.array([self.m.jnt_qposadr[name2id(j)] for j in LEG])
        self.vadr = np.array([self.m.jnt_dofadr[name2id(j)] for j in LEG])
        self.gyro = self.m.sensor_adr[mujoco.mj_name2id(self.m, mujoco.mjtObj.mjOBJ_SENSOR,
"imu_ang_vel")]
        self.cmd = np.array([vx, 0.0, 0.0])
        self.dt = self.m.opt.timestep * DECIMATION
        self.reset()

    def reset(self):
        mujoco.mj_resetData(self.m, self.d)
        allq = np.concatenate([DEFAULT_LEG, ARM_HOLD])
        adr = [self.m.jnt_qposadr[mujoco.mj_name2id(self.m, mujoco.mjtObj.mjOBJ_JOINT, n)]
               for n in [j + "_joint" for j in LEG] +
               ["waist_roll_joint", "waist_yaw_joint",
                "left_shoulder_pitch_joint", "left_shoulder_roll_joint",
"left_shoulder_yaw_joint",
                "left_elbow_joint", "left_wrist_roll_joint",
                "right_shoulder_pitch_joint", "right_shoulder_roll_joint",
"right_shoulder_yaw_joint",
                "right_elbow_joint", "right_wrist_roll_joint"]]
        self.d.qpos[adr] = allq
        mujoco.mj_forward(self.m, self.d)
        self.prev_action = np.zeros(12)
        self.buf = []

    def obs(self, sl=None, sigma=None, mult=0.0, rng=None, zero_sl=None, zero_val=None):
        d = self.d
        angvel = d.sensordata[self.gyro:self.gyro+3].copy()
        grav = projected_gravity(d.qpos[3:7])
        dofpos = d.qpos[self.qadr] - DEFAULT_LEG
        dofvel = d.qvel[self.vadr].copy()
        o = np.concatenate([angvel*S_ANGVEL, grav, self.cmd, dofpos, dofvel*S_DOFVEL,
self.prev_action])
        if sl is not None:
            o[sl] += rng.standard_normal(sl.stop - sl.start) * sigma * mult
        if zero_sl is not None:
            o[zero_sl] = zero_val          # pin a channel to its TRAINING value
        return o

    def step(self, action):
        self.prev_action = action.copy()
        target = DEFAULT_LEG + action * ACTION_SCALE
        self.d.ctrl[:] = np.concatenate([target, ARM_HOLD])
        for _ in range(DECIMATION):
            mujoco.mj_step(self.m, self.d)

    def tilt(self):
        z = self.d.xmat[1].reshape(3, 3)[:, 2]
        return np.degrees(np.arccos(np.clip(z[2], -1, 1)))


def run(a):
```

```
    xml = a.xml or os.path.join(HERE, "model", "r1_walk_train.xml")
    pol_path = a.policy if os.path.isabs(a.policy) else os.path.join(HERE, "policies", a.policy)
    act = load_policy(pol_path)
    stats = np.load(pol_path)
    r = Robot(xml, a.vx)
    rng = np.random.default_rng(a.seed)

    sl = sigma = mult = None
    if a.noise:
        sl, sig, scale = CH[a.noise[0]]
        mult = float(a.noise[1])
        # a channel with injected training noise is dosed in multiples of it;
        # one without is dosed in multiples of its own training spread
        sigma = (np.full(sl.stop - sl.start, sig * scale) if sig is not None
                 else np.sqrt(stats["obs_var"][sl]))
    zero_sl = CH[a.zero][0] if a.zero else None
    zero_val = stats["obs_mean"][zero_sl] if a.zero else None

    label = [os.path.basename(pol_path)]
    if a.vx:      label.append(f"vx {a.vx}")
    if a.xml:     label.append(f"model {os.path.basename(xml)}")
    if a.push:    label.append(f"push {a.push}")
    if a.latency: label.append(f"latency {a.latency*20} ms")
    if a.noise:   label.append(f"{a.noise[0]} noise x{a.noise[1]}")
    if a.zero:    label.append(f"{a.zero} zeroed")
    print(" | ".join(label))

    def one_frame(s):
        o = r.obs(sl, sigma, mult, rng, zero_sl, zero_val)
        r.buf.append(act(o))
        used = r.buf[max(0, len(r.buf) - 1 - a.latency)]     # stale action = control delay
        if a.push and s == int(1.0 / r.dt):
            r.d.qvel[0] += a.push
        r.step(used)

    total, peak = int(a.seconds / r.dt), 0.0
    if a.headless:
        for s in range(total):
            one_frame(s)
            peak = max(peak, r.tilt())          # the PEAK, not the tilt it happens
            if r.tilt() > 45:                   # to be sitting at when time runs out
                print(f"  fell at {s*r.dt:.2f}s"); return
        print(f"  STOOD the full {a.seconds:g}s  (max tilt {peak:.2f} deg)"); return

    import mujoco.viewer
    with mujoco.viewer.launch_passive(r.m, r.d) as v:
        v.cam.distance, v.cam.azimuth, v.cam.elevation = 3.0, 135, -8
        s, done = 0, None
        while v.is_running():
            t0 = time.time()
            if s < total and done is None:
                one_frame(s); s += 1
                peak = max(peak, r.tilt())
                if r.tilt() > 45:
```

```
                    done = f"fell at {s*r.dt:.2f}s"; print("  " + done)
                elif s >= total:
                    done = f"STOOD the full {a.seconds:g}s  (max tilt {peak:.2f} deg)"
                    print("  " + done)
            v.cam.lookat[:] = [r.d.qpos[0], r.d.qpos[1], 0.45]
            v.sync()
            lag = r.dt - (time.time() - t0)
            if lag > 0:
                time.sleep(lag)


if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--policy", default="exp2_stand.npz")
    p.add_argument("--xml", default=None)
    p.add_argument("--vx", type=float, default=0.0)
    p.add_argument("--seconds", type=float, default=12.0)
    p.add_argument("--push", type=float, default=0.0)
    p.add_argument("--latency", type=int, default=0, help="control delay in 20 ms steps")
    p.add_argument("--noise", nargs=2, metavar=("CHANNEL", "MULT"),
                   help="corrupt one channel: angvel|gravity|dofpos|dofvel|command|prev_action")
    p.add_argument("--zero", metavar="CHANNEL",
                   help="pin a channel to its training value -- the NULL CONTROL")
    p.add_argument("--seed", type=int, default=0)
    p.add_argument("--headless", action="store_true")
    run(p.parse_args())
```

```
python watch_policy.py --headless
```

Expected:

```
exp2_stand.npz
  STOOD the full 12s  (max tilt 3.35 deg)
```

Now the thing Lab 1 could not do. In Lab 1 the hand-tuned controller survived a 0.15 m/s shove and fell at 0.20:

```
python watch_policy.py --push 0.30 --headless
```

Expected:

```
exp2_stand.npz | push 0.3
  STOOD the full 12s  (max tilt 4.98 deg)
```

```
python watch_policy.py --push 0.40 --headless
```

Expected:

```
exp2_stand.npz | push 0.4
  fell at 2.06s
```

Twice the shove Lab 1 could take, and you can see where it stops.

7b — Look at it *(works everywhere, including WSL)*

Create `render_policy.py`:

```
"""Render still pictures of a TRAINED policy -- Labs 2 and 3, WSL-safe.

MuJoCo's interactive viewer fails under WSL2 (gladLoadGL), so watch_policy.py's
window is unavailable to a lot of students.  Offscreen rendering works
everywhere, so this draws the same rollout as pictures: same Robot, same policy
loader, same 50 Hz control loop as watch_policy.py -- only the output differs.

    python render_policy.py                       # the four Lab 2 pictures
    python render_policy.py --push 0.4 --at 5 --out fell.png
"""
import os, sys
# WSL2 has no usable EGL; osmesa (software GL) is the backend that works here.
# Must be set before MuJoCo creates a GL context.
os.environ.setdefault("MUJOCO_GL", "osmesa")
import numpy as np, mujoco, imageio

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
from watch_policy import Robot, load_policy          # noqa: E402

W, H = 900, 700


def shot(out, policy="exp2_stand.npz", push=0.0, at_s=5.0, vx=0.0, xml=None,
         label=""):
    xml = xml or os.path.join(HERE, "model", "r1_walk_train.xml")
    path = policy if os.path.isabs(policy) else os.path.join(HERE, "policies", policy)
    act = load_policy(path)
    r = Robot(xml, vx)
    steps = int(at_s / r.dt)
    push_step = int(1.0 / r.dt)
    peak = 0.0
    for s in range(steps):
        if push and s == push_step:
            r.d.qvel[0] += push
        r.step(act(r.obs()))
        peak = max(peak, r.tilt())
    # The stock scene renders almost black on a projector.  Raise the headlight
    # so the robot is actually visible in a lit room.
    r.m.vis.headlight.ambient[:] = [0.6, 0.6, 0.6]
```

```
    r.m.vis.headlight.diffuse[:] = [0.7, 0.7, 0.7]
    r.m.vis.headlight.specular[:] = [0.1, 0.1, 0.1]
    cam = mujoco.MjvCamera()
    mujoco.mjv_defaultCamera(cam)
    cam.distance, cam.azimuth, cam.elevation = 2.7, 135, -8
    cam.lookat[:] = [r.d.qpos[0], r.d.qpos[1], 0.5]
    # default offscreen framebuffer is 640x480; raise it before the renderer
    r.m.vis.global_.offwidth = max(r.m.vis.global_.offwidth, W)
    r.m.vis.global_.offheight = max(r.m.vis.global_.offheight, H)
    ren = mujoco.Renderer(r.m, height=H, width=W)
    try:
        ren.update_scene(r.d, camera=cam)
        img = ren.render()
    finally:
        ren.close()
    imageio.imwrite(out, img)
    print(f"{os.path.basename(out):28s} push={push:<5g} t={at_s}s  "
          f"pelvis_h={r.d.xpos[1][2]:.3f}m  max_tilt={peak:5.1f} deg  {label}")


if __name__ == "__main__":
    import argparse
    p = argparse.ArgumentParser()
    p.add_argument("--policy", default="exp2_stand.npz")
    p.add_argument("--push", type=float, default=None)
    p.add_argument("--at", type=float, default=5.0)
    p.add_argument("--vx", type=float, default=0.0)
    p.add_argument("--xml", default=None)
    p.add_argument("--out", default=None)
    a = p.parse_args()
    if a.out or a.push is not None:
        shot(a.out or "shot.png", a.policy, a.push or 0.0, a.at, a.vx, a.xml)
    else:
        o = os.path.join(HERE, "lab_img")
        os.makedirs(o, exist_ok=True)
        shot(os.path.join(o, "01_policy_start.png"), a.policy, 0.0, 0.02)
        shot(os.path.join(o, "02_policy_standing.png"), a.policy, 0.0, 5.0)
        shot(os.path.join(o, "03_push_030_survived.png"), a.policy, 0.30, 5.0)
        shot(os.path.join(o, "04_push_040_fell.png"), a.policy, 0.40, 5.0)
```

```
python render_policy.py
```

Expected:

```
01_policy_start.png          push=0     t=0.02s  pelvis_h=0.739m  max_tilt=  1.3 deg
02_policy_standing.png       push=0     t=5.0s  pelvis_h=0.718m  max_tilt=  3.4 deg
03_push_030_survived.png     push=0.3   t=5.0s  pelvis_h=0.718m  max_tilt=  5.0 deg
04_push_040_fell.png         push=0.4   t=5.0s  pelvis_h=0.086m  max_tilt= 95.4 deg
```

Four PNGs in `lab_img/`. Open them in the VS Code sidebar. Compare `03` (pelvis at 0.718 m, tilted 5°, five seconds after a shove that would have felled Lab 1's robot) with `04` (0.086 m, 95° — flat on the floor).

Note that `02` and `03` end at the **same** pelvis height, 0.718 m. Four seconds after the push it is not merely upright, it is back where it started.

7c — Watch it live *(needs a working OpenGL window)*

```
python watch_policy.py
```

A window opens and the robot stands. Left-drag orbits, scroll zooms, `Esc` quits. Drop `--headless` from any command in 7a to watch that case instead.

> ⚠ `ERROR: gladLoadGL error` **under WSL2 is expected on some machines and is not your mistake** — the same warning as Lab 1 Part 9b. If you hit it, 7c is unavailable to you; 7a gives you the verdict and 7b gives you the pictures, which is the whole content of this part.

Part 8 — Does it stand?

Watching is not measuring. Create `exp2_eval.py`:

```
"""Experiment 2 evaluation -- what did the standing policy actually learn?

Three questions the training curve cannot answer:

  1. Does it stand?  ep_rew_mean says nothing about whether the episode ended
     at the 20 s cap or on a fall.
  2. Can it take a push?  Experiment 1's PD controller survived 0.15 m/s and
     fell at 0.20 m/s.  The push protocol here is deliberately IDENTICAL --
     a base velocity impulse injected at t = 1 s -- so the two numbers are
     directly comparable.  Anything else would be comparing different tests.
  3. What is it being paid for?  The reward is a sum of terms; without
     decomposing it, a high score could just be an alive bonus being farmed.

Randomization is OFF (dr=False, push=False) so the only disturbance is the one
we inject.
"""
import argparse
import os
from collections import defaultdict

import numpy as np
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

from r1_walk_env import R1WalkEnv
```

```
HERE = os.path.dirname(os.path.abspath(__file__))


def make(name, ckpt, vn):
    d = os.path.join(HERE, "runs", name)
    model = PPO.load(os.path.join(d, ckpt), device="cpu")
    raw = R1WalkEnv(push=False, dr=False, fixed_cmd=[0.0, 0.0, 0.0], seed=None)
    venv = DummyVecEnv([lambda: raw])
    venv = VecNormalize.load(os.path.join(d, vn), venv)
    venv.training = False
    venv.norm_reward = False
    return model, venv, raw


def episode(model, venv, raw, push=0.0, push_at_s=1.0):
    """One episode.  Returns metrics plus the per-term reward totals."""
    obs = venv.reset()
    terms = defaultdict(float)
    h0 = raw.data.xpos[1][2]
    min_h = h0
    max_tilt, steps, total_r = 0.0, 0, 0.0
    push_step = int(push_at_s / raw.dt)
    fell = False

    while True:
        if push and steps == push_step:
            raw.data.qvel[0] += push          # same protocol as experiment 1

        # Read state BEFORE step().  DummyVecEnv auto-resets on done, so reading
        # raw.data after the loop measures a FRESH episode -- which reported a
        # -0.38 cm "height drop" for episodes that had actually toppled.  Fixed
        # 2026-08-14; the tilt figure was never affected (accumulated in-loop).
        min_h = min(min_h, raw.data.xpos[1][2])
        zaxis = raw.data.xmat[1].reshape(3, 3)[:, 2]
        max_tilt = max(max_tilt, np.degrees(np.arccos(np.clip(zaxis[2], -1, 1))))

        act, _ = model.predict(obs, deterministic=True)
        obs, r, done, infos = venv.step(act)
        info = infos[0]
        total_r += float(r[0])
        steps += 1

        for k, v in info.get("rew", {}).items():
            terms[k] += float(v)

        if done[0]:
            # SB3 reports truncation in the info dict; anything else is a real fall
            fell = not info.get("TimeLimit.truncated", False)
            break

    return {"steps": steps, "seconds": steps * raw.dt, "fell": fell,
            "max_tilt": max_tilt, "height_drop_cm": 100 * (h0 - min_h),
            "reward": total_r, "terms": dict(terms)}
```

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--name", default="exp2_stand")
    ap.add_argument("--ckpt", default="best")
    ap.add_argument("--vn", default="vecnormalize_best.pkl")
    ap.add_argument("--episodes", type=int, default=10)
    args = ap.parse_args()

    model, venv, raw = make(args.name, args.ckpt, args.vn)
    cap = raw.max_steps * raw.dt
    print(f"=== Experiment 2 eval: {args.name}/{args.ckpt} "
          f"(episode cap {cap:.0f}s, no DR, no random push) ===\n")

    # ---------- 1. undisturbed standing ----------
    rows = [episode(model, venv, raw) for _ in range(args.episodes)]
    surv = np.array([r["seconds"] for r in rows])
    fell = np.array([r["fell"] for r in rows])
    print(f"--- undisturbed, {args.episodes} episodes ---")
    print(f"  survival        : {surv.mean():5.2f}s +/- {surv.std():4.2f}  "
          f"(min {surv.min():.2f}, max {surv.max():.2f})")
    print(f"  reached the cap : {(~fell).sum()}/{len(rows)}  ({100*(~fell).mean():.0f}%)")
    print(f"  max tilt        : {np.mean([r['max_tilt'] for r in rows]):5.2f} deg")
    print(f"  height drop     : {np.mean([r['height_drop_cm'] for r in rows]):5.2f} cm")
    print(f"  episode reward  : {np.mean([r['reward'] for r in rows]):8.1f}")

    # ---------- 2. what is it paid for ----------
    tot = defaultdict(float)
    for r in rows:
        for k, v in r["terms"].items():
            tot[k] += v / len(rows)
    print(f"\n--- reward decomposition (mean per episode) ---")
    gross = sum(abs(v) for v in tot.values()) or 1.0
    for k, v in sorted(tot.items(), key=lambda kv: -abs(kv[1])):
        print(f"  {k:18s} {v:9.1f}   ({100*abs(v)/gross:4.1f}% of gross)")

    # ---------- 3. push threshold, same protocol as experiment 1 ----------
    print(f"\n--- push rejection (impulse at t=1s; PD baseline was 0.15 ok / 0.20 fail) ---")
    print(f"  {'push m/s':>9}{'survived':>10}{'mean s':>9}{'max tilt':>10}")
    for p in [0.1, 0.15, 0.2, 0.3, 0.4, 0.6, 0.8, 1.0]:
        trials = [episode(model, venv, raw, push=p) for _ in range(5)]
        ok = sum(not t["fell"] for t in trials)
        print(f"  {p:>9}{ok:>7}/5{np.mean([t['seconds'] for t in trials]):>9.2f}"
              f"{np.mean([t['max_tilt'] for t in trials]):>10.1f}")


if __name__ == "__main__":
    main()
```

This one **does** need PyTorch — it loads the training checkpoint rather than the exported `.npz`, which is what lets it read the reward apart term by term in Part 9.

```
python exp2_eval.py --name exp2_stand --ckpt best --episodes 20
```

It takes about two minutes. **Expected — the first block:**

```
=== Experiment 2 eval: exp2_stand/best (episode cap 20s, no DR, no random push) ===

--- undisturbed, 20 episodes ---
  survival        : 20.00s +/- 0.00  (min 20.00, max 20.00)
  reached the cap : 20/20  (100%)
  max tilt        :  3.19 deg
  height drop     :  1.84 cm
  episode reward  :   2103.3
```

20 out of 20, tilting about 3 degrees, sinking under 2 cm. Not merely upright — nearly motionless.

> **Your last three numbers will not match exactly, and should not.** The starting pose is randomised slightly every episode, so across 20-episode runs tilt lands between about **3.1 and 3.4 degrees** and reward between about **2098 and 2110** — with fewer episodes the spread is wider. What must match exactly is `20/20` and `20.00s +/- 0.00`.

> ☑ **Checkpoint: 20/20 reached the cap.**

Part 9 — What is it actually paid for?

The same command printed a decomposition. This is the answer to the objection you raised in Part 6.

Expected — the second block:

```
--- reward decomposition (mean per episode) ---
  lin_vel               1497.4   (68.5% of gross)
  ang_vel                497.1   (22.7% of gross)
  alive                  150.0   ( 6.9% of gross)
  torque                 -28.7   ( 1.3% of gross)
  heading                 -6.8   ( 0.3% of gross)
  base_height             -2.8   ( 0.1% of gross)
  orientation             -1.5   ( 0.1% of gross)
```

The survival bonus is 6.9% of the score. Ninety-one percent comes from velocity tracking — from holding still *on purpose*, which is the actual task. If the policy were farming the alive bonus, those two numbers would be the other way round.

A second check worth doing in your head: `alive` is exactly **150.0 = 0.15 × 1000 steps**. That arithmetic confirms every one of the 20 episodes ran the full length. A single fall would have dented it.

> You raised the objection before seeing the result, then answered it with a measurement. That is the difference between a demo and an experiment.

Part 10 — Push it

Same command, third block. Same protocol as Lab 1 — a velocity impulse at t = 1 s.

Expected:

```
--- push rejection (impulse at t=1s; PD baseline was 0.15 ok / 0.20 fail) ---
   push m/s  survived   mean s  max tilt
        0.1      5/5    20.00       3.4
       0.15      5/5    20.00       3.1
        0.2      5/5    20.00       3.1
        0.3      5/5    20.00       5.3
        0.4      0/5     2.28      58.5
        0.6      0/5     1.85      57.4
        0.8      0/5     1.66      57.7
        1.0      0/5     1.58      56.3
```

	Lab 1 — two dials	Lab 2 — trained policy
survives	0.15 m/s	**0.30 m/s**
fails at	0.20 m/s	**0.40 m/s**

Learning roughly doubled it.

The protocol is copied from Lab 1 exactly. If a nicer test had been invented for the learned policy, the comparison would be worthless. Same robot, same shove, same measurement — only the controller changed.

And the reason is structural, not incremental. Lab 1's controller knows one instruction: *get back to the pose I was told to hold.* Recovering from a shove means choosing a **different** pose, and the target was a constant, so there was nothing to change. The policy picks a new target fifty times a second, so it can crouch, shift its weight, and give ground. **A different kind of thing, not a better-tuned version of the same thing.**

Look at the `max tilt` column too: the failures tilt about 57°, not 91°. They are not falling harder — they are falling from a fight.

> ☑ **Checkpoint: 0.30 survives, 0.40 does not.** Write both next to Lab 1's 0.15 / 0.20.

Part 11 — Back to your own run

Your training has been going since Part 4. Go and look at it:

```
ls runs/my_run/ckpt_*.zip | tail -5
```

Expected — the run this manual was written with, after about 25 minutes:

```
runs/my_run/ckpt_750k.zip
runs/my_run/ckpt_800k.zip
runs/my_run/ckpt_850k.zip
runs/my_run/ckpt_900k.zip
runs/my_run/ckpt_950k.zip
```

Every checkpoint is a policy you can evaluate. **Evaluate several — not just the newest.**

```
python exp2_eval.py --name my_run --ckpt ckpt_1000k --vn vn_1000k.pkl --episodes 6
```

Repeat for two or three others you have. Use `--episodes 6` — you are sampling, not publishing.

Here is what that run gave at 1M steps. Read it before you read your own:

```
=== Experiment 2 eval: my_run/ckpt_1000k (episode cap 20s, no DR, no random push) ===

--- undisturbed, 6 episodes ---
  survival        :  1.65s +/- 0.07  (min 1.56, max 1.78)
  reached the cap : 0/6  (0%)
  max tilt        : 48.13 deg
  height drop     : 30.43 cm
  episode reward  :     42.4
```

It does not stand. Same command, same 1M steps, and the reference run in the table below stood 20 out of 20 at this exact point. Its training reward at 1M was 159; this one's was 73.6.

That is not a broken run and it is not a mistake in this manual. It is the thing this part is about, and you are looking at a real instance of it, not a story about one.

The same run at **2M steps**, an hour in:

```
--- undisturbed, 6 episodes ---
  survival        :  5.81s +/- 3.68  (min 2.52, max 13.54)
  reached the cap : 0/6  (0%)
  max tilt        : 44.73 deg
```

Better — three and a half times the survival — but still not standing, where the reference run stood 20/20 at the same point. **Two runs of the same command, launched on two different machines, ended up in genuinely different places.** Report what your run did; do not report what it was supposed to do.

Watch your own policy, not just its numbers

`exp2_eval.py` gives you survival times. To *see* your own run you need it in the same plain format the reference policy came in. That is what `export_policy.py` does — it pulls the weights and the observation statistics out of a checkpoint and writes the `.npz`. Create it:

```
"""Export a trained PPO policy to a plain .npz so it can run with NumPy alone.

The policy is a 45 -> 256 -> 256 -> 12 MLP with tanh activations, plus the
VecNormalize observation statistics. None of that needs PyTorch to evaluate, so
exporting lets the student labs keep the same light dependency as Lab 1
(mujoco + numpy) instead of pulling in a ~2.5 GB torch install.

    python export_policy.py runs/exp2_stand best vecnormalize_best.pkl out.npz
"""
import sys, os
import numpy as np
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
import gymnasium as gym


def export(run_dir, ckpt, vn_name, out):
    model = PPO.load(os.path.join(run_dir, ckpt), device="cpu")
    sd = model.policy.state_dict()
    g = lambda k: sd[k].cpu().numpy()

    class _Stub(gym.Env):
        observation_space = model.observation_space
        action_space = model.action_space
        def reset(self, **kw): return np.zeros(model.observation_space.shape, np.float32), {}
        def step(self, a): return np.zeros(model.observation_space.shape, np.float32), 0.0,
False, False, {}

    vn = VecNormalize.load(os.path.join(run_dir, vn_name), DummyVecEnv([lambda: _Stub()]))

    np.savez(
        out,
        w0=g("mlp_extractor.policy_net.0.weight"), b0=g("mlp_extractor.policy_net.0.bias"),
        w1=g("mlp_extractor.policy_net.2.weight"), b1=g("mlp_extractor.policy_net.2.bias"),
        w2=g("action_net.weight"), b2=g("action_net.bias"),
        obs_mean=vn.obs_rms.mean.astype(np.float64),
        obs_var=vn.obs_rms.var.astype(np.float64),
        clip_obs=np.float64(vn.clip_obs), epsilon=np.float64(vn.epsilon),
        act_low=model.action_space.low.astype(np.float64),
        act_high=model.action_space.high.astype(np.float64),
    )
    print(f"wrote {out}  ({os.path.getsize(out)/1024:.0f} KB)")
    return model, vn
```

```
def numpy_policy(npz):
    """The whole inference path, in NumPy. This is what the labs will ship."""
    d = np.load(npz)
    def act(obs):
        o = np.clip((obs - d["obs_mean"]) / np.sqrt(d["obs_var"] + d["epsilon"]),
                    -d["clip_obs"], d["clip_obs"])
        h = np.tanh(d["w0"] @ o + d["b0"])
        h = np.tanh(d["w1"] @ h + d["b1"])
        # SB3's predict() clips the action to the action space -- without this the
        # numpy policy disagrees by up to 4.6 on saturated joints.
        return np.clip(d["w2"] @ h + d["b2"], d["act_low"], d["act_high"])
    return act


if __name__ == "__main__":
    run_dir, ckpt, vn_name, out = sys.argv[1:5]
    model, vn = export(run_dir, ckpt, vn_name, out)

    # verify: numpy must reproduce sb3's deterministic action exactly
    act = numpy_policy(out)
    rng = np.random.default_rng(0)
    worst = 0.0
    for _ in range(200):
        raw = rng.normal(0, 1.5, size=model.observation_space.shape[0]).astype(np.float64)
        norm = np.clip((raw - vn.obs_rms.mean) / np.sqrt(vn.obs_rms.var + vn.epsilon),
                       -vn.clip_obs, vn.clip_obs).astype(np.float32)
        ref, _ = model.predict(norm[None, :], deterministic=True)
        worst = max(worst, float(np.abs(act(raw) - ref[0]).max()))
    print(f"max |numpy - sb3| over 200 random observations: {worst:.3e}")
    print("MATCH" if worst < 1e-5 else "MISMATCH — do not ship")
```

```
python export_policy.py runs/my_run ckpt_1000k vn_1000k.pkl policies/my_run.npz
```

Expected:

```
wrote policies/my_run.npz  (319 KB)
max |numpy - sb3| over 200 random observations: 4.138e-07
MATCH
```

The export is **checked against the real network** on 200 random observations before it is trusted — that is what `MATCH` means. Your last digits will differ; the exponent should not. An export that silently disagreed with the network would make every picture you draw from it a lie.

Then watch it exactly as you watched the reference in Part 7:

```
python watch_policy.py --policy my_run.npz --headless
```

```
python render_policy.py --policy my_run.npz --push 0.30 --out my_push.png
```

Expected — again, from the run this manual was written with:

```
my_run.npz
  fell at 1.42s
```

```
my_push.png                 push=0.3   t=5.0s  pelvis_h=0.037m  max_tilt= 79.7 deg
```

Open `my_push.png` beside Part 7's `03_push_030_survived.png`. Same shove, same robot, same command that produced both policies — one is standing at 0.718 m, the other is on the floor at 0.037 m.

Use the same checkpoint number in all of these. Comparing your 1M against the reference's 1M is a comparison; comparing your 1M against its 3M is not.

> **Expect your numbers to differ from the reference, and do not treat that as a mistake.** Two runs of the identical command with different seeds reached 70 and 159 reward at the same 1M steps — one less than half the other. Which checkpoints work, and whether you see the collapse below at all, is genuinely run-dependent.

Why several, and not just the last one

Here is what the reference run does, measured checkpoint by checkpoint:

checkpoint	training reward	survival	reached cap
500k	19	11.52 s	0/10
1M	159	**20.00 s**	**6/6** ☑
1.45M	342	1.47 s	0/8 ❌
1.5M	**417 ← best so far**	**1.46 s**	**0/8** ❌
1.65M	**436 ← new best**	1.57 s	0/8 ❌
2M	617	20.00 s	6/6 ☑
3M	908	20.00 s	6/6 ☑

It works at 1M, **is completely broken from about 1.45M to 1.65M**, and recovers by 2M.

And the training curve never shows it. Reward rises the whole way through — hitting an all-time best at 1.5M and again at 1.65M — while the policy cannot stand for two seconds.

> **The lesson of this lab**
>
> **A rising training curve does not mean your policy works.** Training reward is measured with randomisation and random pushes ON, averaged over recent episodes. Evaluation runs with them OFF. They are different quantities, and here they disagree completely.
>
> The only way to know whether a policy works is to **evaluate it**.

This is the third time this workshop has met the same shape of problem. Lab 1 had a `NaN` check that could not see a blown-up simulator, and a fall detector that could not see the robot circling. All three are **a number that looks like the thing you care about, but is not.**

If your own run landed in a rough patch, you have reproduced a real research finding. Say so.

Troubleshooting

Symptom	Cause & fix
`ModuleNotFoundError: stable_baselines3` or `torch`	Part 0 did not finish. Parts 5–7 do not need them
The torch download is ~4.8 GB	you got the CUDA build — use the `--index-url` line in Part 0
`ModuleNotFoundError: gymnasium`	it comes with stable-baselines3; Part 0 did not finish
`resource not found ... .STL`	`model/assets` is missing — rerun the `cp -r` in Part 1
`XML Error` on `r1_walk_train.xml`	the paste is truncated. `wc -l model/r1_walk_train.xml` must say **390**
12 actuators instead of 24	you pasted the new block without deleting the old one
Training prints nothing for minutes	normal — one line per 50,000 steps, about 75 s apart
`ckpt_1000k.zip not found`	your run has not reached 1M yet. Use one it has: `ls runs/my_run`
`ERROR: gladLoadGL error`	no usable OpenGL window. Expected under WSL2 — use 7a and 7b
Your policy evaluates badly	**read Part 11 before assuming you broke it**
Numbers differ from this manual	tell the instructor — a finding

What to hand in

1. The first three lines of your training output — including the negative starting reward.
2. Your `diff` from Part 2, showing that the only change to the robot was its actuators.
3. Your evaluation of the reference policy: survival, tilt, and the alive-bonus percentage.
4. Your push result, next to Lab 1's 0.15 / 0.20.
5. **Evaluations of at least three of your own checkpoints**, and what you conclude.
6. One paragraph: *why can a learned policy recover from a shove when a PD controller cannot?*

7. One sentence: *what does a rising training curve prove?*

Appendix A — Reward Tuning at Home (optional)

You now know how to launch training. Changing the reward is one edit away — but the effects need a lot more compute than a lab session, so this is homework.

Open `r1_walk_env.py` and find `RW`, `TARGET_H` and `TARGET_STANCE_WIDTH`.

Two kinds of parameter, and they behave completely differently

Kind	Examples	When you see the effect
Target-shaping	`TARGET_H`, `TARGET_STANCE_WIDTH`, the command	**fast** — often within 1M steps
Tradeoff weights	`orientation`, `torque`, `feet_air_time`, `heading`	**slowly, if at all**

Target-shaping changes *what pose is being asked for*, so behaviour changes visibly. Weight changes only shift a balance between competing pressures, and the policy may end up in much the same place.

Start with `TARGET_H`. Change 0.65 to 0.50 and train 1M steps — you should get a visibly more crouched robot in about 25 minutes.

What it costs to do properly

One run is 3M steps ≈ 74 minutes. A real comparison needs several conditions and several seeds:

Design	Runs	Wall time
3 conditions × 2 seeds × 1M	6	~2.5 h
3 conditions × 3 seeds × 3M	9	~11 h
5 conditions × 3 seeds × 3M	15	~18 h

Overnight is the realistic option.

An honest warning

This exact experiment has been run on this project — five reward conditions, three seeds, 800k steps each. The verdict was **seed variance exceeds the term effects at this compute.** No causal claim about any single reward term was supportable from fifteen runs.

So if you change a penalty weight, train overnight, and see no clean difference: **that is the expected result, not a broken run.** "At laptop compute, reward-term effects are below seed noise" is a legitimate finding, and a more honest one than a difference invented from a single seed.

Appendix B — Instructor Notes

- **Part 4 must happen early.** The whole session design depends on training accumulating in the background. If a student is late, hand them a partial run on a USB stick.
- **Parts 5–7 need no PyTorch.** If a student's install failed, they can still read the environment, read the reward, and watch the reference policy. Parts 8–11 need it. Do not let an install failure block the first hour.
- **The one download is unavoidable.** A trained policy cannot be typed. `lab2_policy.tar.gz` is 2 MB: the `.npz` for the no-PyTorch viewer, and the SB3 checkpoint plus its normalisation statistics for the evaluator. Everything else in the lab the student builds.
- **`tar`, not `unzip`.** A clean Ubuntu 24.04 under WSL2 has no `unzip`.
- **Part 2 is the conceptual hinge**, and it is 26 lines of paste rather than 390 because the Lab 2 model *is* the Lab 1 model with a different actuator block. Students should run the `diff` themselves — seeing that nothing else moved is what makes the later sim2sim comparison meaningful.
- **Part 11 is the payload.** The 1.45M–1.65M collapse is real, measured, and invisible in the training curve. Students who land in it have reproduced a genuine research finding.
- **Expect "my policy is worse than yours."** Correct answer: possibly, and possibly not — evaluate more checkpoints.
- **Timing:** Parts 0–4 about 30 min (most of it pasting and the first install), Parts 5–7 fifteen, Parts 8–10 fifteen, Part 11 ten. Training reaches ~1.5–2M steps in that window.
- Reward tuning is deliberately homework. It cannot produce a trustworthy result in a session, and Appendix A says so plainly rather than implying otherwise.